

Project Alpha — Checkpoint Update 3

Coverage model, reward design, and architectural pins · June 2026

This checkpoint covers the coverage-model rework (bool→float scan-progress grid), the reward function design, and a series of architectural decisions on dwell semantics, map sharing, three-layer observation, and rejected alternatives. Read alongside Checkpoints 1 and 2.

1. Progress Snapshot

Module	Status	Tests
coverage_map.py	DONE (float rework)	17/17
mid_level_controller.py	DONE (geofence + seed)	16 + 2 known
observation.py	DONE	13/13
reward.py	DONE	12/12
drone_env.py (DirectRLEnv)	NEXT	—
train.py	Pending	—

All four core computation modules are built, tested, and verified in isolation. The next step is drone_env.py which wires them together as an Isaac Lab DirectRLEnv.

2. Coverage Model — Bool to Float Rework

2.1 What changed

The coverage grid changed from `torch.bool` (binary: scanned or not) to `torch.float` (scan-progress 0.0→1.0). A cell is not instantly scanned on contact; it requires sustained drone presence (dwell) to accumulate progress toward completion.

Aspect	Before (bool)	After (float)
Grid dtype	bool	float [0.0, 1.0]
mark_scanned	OR-ing True, instant	increment by scan_rate, clamp at 1.0
'Covered' cell	True	progress >= threshold (default 0.95)
Multi-agent same cell	OR (either marks it)	MAX (presence is presence, not additive)
Reward signal	sparse (cell flips)	dense (every step of progress earns reward)
Buffer	_batch_mask (uint8, OR)	_batch_progress (float, index_reduce_ amax)
mark_scanned returns	nothing	(progress_delta, completion_count) per env

2.2 scan_rate and dwell semantics

scan_rate = 1 / dwell_steps. With scan_rate=0.2, a cell needs 5 steps of continuous drone presence to reach 1.0. This is the actual dwell-time knob. When tuning, think in dwell-steps (how many steps of presence to complete a cell), not in abstract rate — 'scan_rate 0.2' obscures what it physically means ('5 steps of presence').

2.3 What the float means (and doesn't mean)

The float is a dwell counter, not a physical model. '0.5 scanned' has no spatial meaning — it does not mean 'the left half of the cell is observed.' There is no sub-cell structure; the cell is atomic (= sensor footprint). The float serves exactly two purposes: (a) providing a dense reward gradient so the drone learns 'dwelling produces reward' instead of facing sparse 'nothing, nothing, jackpot'; and (b) the drone's own awareness of 'am I done dwelling here yet' via the observation. It is a training device, not a physics simulation.

3. Dwell Model & Map Sharing (Phase 1 Decision)

3.1 Per-drone dwell (Model 2) — locked

Cell scan-progress is **private to each drone**. A cell completes when **one drone** saturates it (reaches threshold). Only **completed cells** (binary, $\geq \text{threshold}$) are shared between drones via the gossip map. Partial float values are never shared — they have no meaning to other drones.

Consequence: two drones cannot cooperate to scan a single cell faster. If drone A dwells 3 steps on a cell and leaves, drone B must dwell the full 5 steps itself — A's partial progress is A's private training signal, not team-usable work. This is a deliberate scope decision: coordinated co-scanning adds complexity (whose dwell counts? how to merge?) for a behavior explicitly declared out of scope.

3.2 Three models considered

Model	Float meaning	Sharing	Status
1 — Shared accumulation	shared dwell counter	partial shareable	current Phase 0 code (fine for single-drone)
2 — Per-drone dwell	private dwell counter	only completed cells	LOCKED for Phase 1+
3 — Directional sweep	physical (left 60% = done)	resumable	REJECTED

Model 3 rejected: directional standard-scan (always scan left-to-right within a cell) would give the float genuine shared physical meaning and let drones resume each other's partial scans. However, its value is gated behind sub-cell sweep control — the drone must actually execute directional sweeps within cells, which is the sub-cell area-coverage problem. Without sweep control the directional convention is unenforceable: a center-hovering drone claiming 'left 60% scanned' is meaningless. Sub-cell sweep is a research problem of its own and explicitly out of scope.

3.3 Phase 0 vs Phase 1 implication

Phase 0 (single drone): Models 1 and 2 are identical — one drone's progress is trivially both shared and private. Current code (Model 1, shared accumulation) works correctly. No change needed. Phase 1: rework mark_scanned so progress is tracked per-drone. Grid becomes (num_envs, num_agents, grid, grid) for private progress, with a separate binary shared overlay for completed cells.

4. Three-Layer Map Architecture (Observation)

4.1 The problem it solves

A single map representation cannot simultaneously provide global routing awareness AND local cell-level precision at variable arena scales. A global pooled map (fixed 10×10) loses local detail for large arenas; a local window loses global awareness of distant unscanned regions. Neither alone is sufficient.

4.2 The three layers

Layer	Source	Representation	Purpose	Atomic?
Coverage grid	ground truth	float [0,1] per cell, scales with area	accounting	yes (it IS the cells)
Global obs (10×10)	binary completed map, percent	percentage completed per region	which region to head to	no — coarse routing

Local obs (11x11)	own float grid, raw cells	raw cell progress, no pooling	which cell to finish	yes — drone-relative
-------------------	---------------------------	-------------------------------	----------------------	----------------------

The coverage grid scales honestly with arena size (more area = more cells). The observation maps are fixed-dimension regardless of arena (global via adaptive pooling, local via fixed cell-count window). The policy always sees $(10 \times 10 + 11 \times 11) = 221$ map values, whether the arena is 50m or 500m.

4.3 Phase 0 scope

At 50m arena with 5m cells, the 10×10 grid IS cell-resolution — the global map is the raw grid (pooling $10 \times 10 \rightarrow 10 \times 10$ is identity). No information loss, no coarseness. Local window stays zeroed. The three-layer distinction only becomes load-bearing at larger arenas where the global map is genuinely coarser than the grid.

5. Observation Vector — Final Locked Layout (283 dims)

5.1 Budget field normalization — arena_diagonal

Both budget fields are normalized by $\text{arena_diagonal} = \sqrt{2} \times \text{arena_size}$ (~70.7m for 50m arena) rather than per-drone `max_range`. This makes them drone-independent (heterogeneous drones see consistent spatial scale) and commensurable (both in the same units, directly subtractable to derive `usable_budget`). A drone's larger `max_range` shows up as a larger `remaining_budget_ratio` numerator, which is exactly the right signal.

```
remaining_budget_ratio = (max_range - odometer) / arena_diagonal
dist_to_base_ratio = dist_to_base_xy / arena_diagonal
usable_budget = remaining - dist (policy learns this itself)
```

5.2 Full layout

Block	Dims	Contents	Phase 0 status
ego	14	pos(3) vel(3) heading_sincos(2) mode_onehot(4) remaining_budget_ratio(1) dist_to_base(1)	live
full_map	100	10x10 coverage (raw float Phase 0, pooled binary Phase 1+)	active
local_window	121	11x11 (raw atomic cells Phase 2+)	zeroed
neighbor_slots	44	K=4 x M=11: rel_pos(3) dist(1) rel_vel(3) mode_onehot(4)	zeroed
validity_mask	4	K=4 neighbor validity	zeroed
TOTAL	283		

5.3 Gating — per-field, at env boundary (unchanged from CP2)

Mode	Ego	Scan fields (maps)	Why
ACTIVE	live	live	normal operation
RETURNING	live	sentinel 1.0	critic sees the drone, not its stale scan data
DOCKED	live	sentinel 1.0	done scanning, state still relevant
DEAD	zeroed EXCEPT mode one-hot	sentinel 1.0	fully masked, mode preserved for identification

6. Reward Function Design

6.1 Philosophy: minimal-first, weight-gated

All reward terms present in code but only the core terms active in Phase 0. Inactive terms have weight=0 and accept None inputs safely (guarded by weight check). Activate by setting weight nonzero and providing the input. This avoids the multiple-term-balancing trap: isolate 'can the drone learn coverage at all' before layering complexity.

6.2 Term structure

Term	Signal	Sign	Phase 0 weight	Source
coverage_delta	scan-progress added this step (clamped)	+	1.0	mark_scanned → progress_delta
completion_bonus	cells crossing threshold this step	+	0.3	mark_scanned → completion_count
time_penalty	flat per step, ACTIVE drones only	−	0.01	mode check
wall_penalty	raw_target overshoot distance (ACTIVE)	−	0 (gated)	controller.raw_target vs clamped
coverage_milestone	env crosses 95% completion	+	0 (gated)	coverage_pct
safe_return_bonus	drone docks after contributing	+	0 (gated)	mode transition
collision_penalty	drone-drone proximity	−	0 (stub)	Phase 1+

6.3 Key design decisions in reward

Time penalty — ACTIVE only

RETURNING/DOCKED drones do not pay the time penalty. The penalty exists to pressure productive scanning urgency. A returning drone is under rule-based control (actor masked); penalizing its time doesn't teach the coverage policy. A docked drone is done. DEAD is forced to 0 entirely.

Completion bonus — carrot, not stick

The completion bonus closes an alignment gap: the dense reward values all progress equally, but the actual metric (coverage_pct) only counts completed cells. The bonus realigns them — it makes the reward structure agree that completion is what counts. At 0.3 per cell (~1.5 dense-steps' worth), it's a noticeable nudge toward finishing, not a new dominant objective.

DEAD masking — in reward, not in loss

DEAD drones are forced to 0 reward inside compute(). RETURNING/DOCKED keep honest reward (which is naturally ~0 since they aren't scanning). Actor/critic loss masking is applied in the training loop, NOT in reward.py — reward computes the honest signal, the loop decides what to learn from.

6.4 Reward range

Per-step reward range with Phase 0 weights: roughly [−0.01, ~+1.1]. The upper bound exceeds +1.0 when a drone progresses multiple cells AND completes one in the same step (corner case with scan_range ≈ cell_size). PPO normalizes advantages, so this is fine. No reward clamping.

7. Rejected Alternatives (With Reasoning)

These were considered, argued through, and explicitly rejected. Recorded so they aren't re-proposed later without remembering the reasoning.

7.1 Abandonment penalty (leaving cell mid-scan)

Rejected. The float value at intermediate states (e.g. 0.5) has no physical meaning under atomic cells — penalizing departure at '0.5' penalizes against a fiction. Cross-timestep detection (which cell was I scanning? for how long?) makes the reward non-Markovian on the observation. Risks: timid scanning (starting a cell is risky if leaving costs), sunk-cost behavior (over-committing to a low-value cell), conflict with the return objective (must leave to return, penalty fights that). The dense reward already incentivizes completion (each dwell step earns reward), and the time penalty captures the efficiency cost of abandon-and-return.

7.2 Loiter countdown penalty (timer after cell completes)

Rejected. Proposed: after a cell completes, start a 5-step countdown, then penalize staying. Problems: requires per-drone temporal state (cross-timestep memory the policy can't see), creates an arbitrary cliff at step 6, and the behavior it targets (lingering on done cells) is already handled by opportunity cost — a done cell yields 0 coverage while fresh cells yield +0.2/step, creating a ~0.2/step gradient toward leaving without any timer. If loitering is observed in training, raise `time_penalty` (global, memoryless) rather than adding a stateful countdown.

7.3 Negative reward for staying on scanned cell

Rejected (related to 7.2). Creates escape pressure (leave fast, any direction) rather than optimization pressure (leave toward the best target). Escape is greedy and direction-blind. Opportunity cost creates pull toward productive cells, which is directional and weighs travel cost. If loitering pathology appears, raising `time_penalty` is the safer lever — it strengthens 'don't waste time' globally without creating escape-direction bias.

7.4 Model 3 directional sweep (covered in §3.2)

Rejected. Sub-cell sweep control required to enforce the convention. See §3.2 for full reasoning.

8. Reward Shaping Philosophy (Pattern)

A recurring pattern emerged through the rejected alternatives, worth recording as a principle:

Shape with carrots and opportunity cost. Observe real failures. Add penalties only against confirmed pathologies. Never add reward terms that need memory the policy can't see.

Specifically: (1) reward positive actions (coverage, completion) rather than penalizing negative ones (abandonment, loitering); (2) let opportunity cost do the work of 'move away from done cells' rather than explicit penalties; (3) if a pathology appears in training, address it with the simplest memoryless lever (raise `time_penalty`) before creating stateful reward terms; (4) any reward term that requires cross-timestep state the policy can't observe makes the reward non-Markovian and should be avoided.

9. Controller Updates This Session

Two additions to `mid_level_controller.py` beyond Checkpoint 2:

- **Geofence clamp:** arena_min/arena_max parameters added. Target xy clamped into bounds before velocity computation. self.raw_target stores pre-clamp target for wall-penalty reward. Out-of-bounds is now a clamp-not-death decision (revised from CP2).
- **dist_to_base seeded in seed_positions:** xy-only distance to base computed from the real spawn position, so observations built before the first step() report correct distance (not stale/zero).

10. Phase 1+ Pins (Comprehensive, Cumulative)

All deferred decisions across Checkpoints 1–3, consolidated. Green boxes are load-bearing architectural decisions; bullet points are smaller items.

Per-agent belief maps (from CP2). The shared per-env coverage grid is a Phase 0-1 simplification. Under comm range limits, each agent maintains its own belief map. `get_full_map` / `get_coverage_pct` become per-env-per-agent. Grid shape $(\text{num_envs}, \text{grid}, \text{grid}) \rightarrow (\text{num_envs}, \text{num_agents}, \text{grid}, \text{grid})$.

Per-drone dwell model (Model 2, this session). Progress private per drone; only completed cells shared via binary gossip map. Current Phase 0 code is shared-accumulation (Model 1), identical for single-drone. Rework `mark_scanned` for per-drone progress when building Phase 1 sharing.

Local window rework (this session). Switch from adaptive-pooled (current) to raw 11×11 atomic-cell window. Drop adaptive pooling for local; keep it only for the global map. Pad out-of-bounds with 1.0. Activate when variable-arena needs the local vs global distinction.

Global map awareness / routing (this session). At large arena scale, the coarse global map may not provide sufficient routing signal toward distant unscanned regions. Approaches to try in order: (1) raw global heatmap + CNN encoder, trust policy; (2) frontier-based encoding (scanned/unscanned boundary) — the established robotics answer. Do NOT add a 'nearest unscanned' vector — it dominates and kills strategic exploration (rejected in design sessions).

Reactive collision-avoidance for RETURNING drones (from CP2 + this session). Rule-based repulsion from nearby drones, applied in the controller after base-ward velocity. RETURNING stays actor-masked for coverage policy. If reactive avoidance proves insufficient, escalate to a separate learned 'get-home-safe' return policy — mode-switched, independently trained.

- **get_local_window optimization** — precompute offset meshgrid in `__init__` when activated (currently dead code).
- **Variable speed + threshold scanning** — reintroduce when sim-to-real realism needed.
- **Battery modeling** — replaces odometer return trigger.
- **Heterogeneous drones** — different `scan_range`/`max_range` per drone; `max_range` becomes per-agent tensor.
- **Dock altitude** — decide whether `RETURN_HEIGHT` docking is final or drones descend to ground.
- **Coverage grid valid-cell mask** — for non-rectangular arena coverage accounting within the rectangular geofence.
- **Per-agent reward credit (Phase 1)** — under Model 2, `mark_scanned` tracks per-drone and returns per-agent deltas. Phase 0 uses per-env→agent mapping.

11. Next — drone_env.py

The final module before training. `DirectREnv` subclass that wires all four tested modules together and interfaces with Isaac Sim. Responsibilities:

- Scene setup: Crazyflie quadrotor asset, ground plane, arena bounds.
- World→local coordinate conversion (the invariant from CP2 §5).

- Reset contract: `controller.reset` → read spawn positions → `controller.seed_positions` → `coverage_map.reset`.
- Step orchestration: policy action → `controller.step` → sim advance → `mark_scanned` → `reward.compute` → `obs_builder.build`.
- Crash detection: NaN/velocity-blowup → `controller.mark_dead` (NOT boundary — boundary is geofence clamp).
- Done conditions: coverage threshold met (`is_fully_covered`), max episode steps, all agents docked/dead.
- Per-env→per-agent delta mapping for `reward.compute`.
- The dual-gate: observation gating (front end) lives here; loss masking (back end) lives in `train.py`.